

Laboratorio 1 — Clúster CockroachDB, localidad y quórum

TI-4601 Bases de Datos Avanzadas · Grupo 3 · Fabricio Mena · Christopher Rodriguez · Alisson Redondo

1. Nodos, localidades, --join y --locality

crdb-1 region=cr-sj SQL 26257 / UI 8080	crdb-2 region=cr-limon sin puertos publicados	crdb-3 region=us-east sin puertos publicados
--	--	---

Los tres con --join=crdb-1,crdb-2,crdb-3 y zone=a; volúmenes crdb1, crdb2, crdb3. Una sola computadora: las tres regiones son localidades lógicas, no una WAN.

--join es la lista con la que los tres procesos se encuentran y forman un solo clúster; no declara regiones. Tras make lab1-up los tres nodos aparecen vivos en node-status-initial.txt, pero SHOW REGIONS FROM DATABASE ti4601 no devuelve nada hasta ejecutar el Paso 3: formar el clúster y configurar la base multi-región son cosas distintas.

--locality es una etiqueta lógica, el par region=...,zone=.... CockroachDB la usa para colocar réplicas y para resolver REGIONAL BY ROW y GLOBAL, pero el nombre no crea distancia: etiqueta no es WAN física. Borrar los volúmenes costaría el almacenamiento estable que Raft supone — término actual, voto del término y log replicado — además de regiones, tablas y datos.

2. Regiones, esquema y semilla

```
ALTER DATABASE ti4601 PRIMARY REGION "cr-sj"; ALTER DATABASE ti4601 ADD REGION "cr-limon";
ALTER DATABASE ti4601 ADD REGION "us-east"; SHOW REGIONS FROM DATABASE ti4601;
\i labs/lab1-cluster/schema.sql \i labs/lab1-cluster/seed.sql \i labs/lab1-cluster/raft_probe.sql
```

Cockroach generó LOCALITY GLOBAL para catalogo_producto y LOCALITY REGIONAL BY ROW AS region para pedido. make lab1-check terminó en 6/6 (config-check.txt).

Decisión 1: region va primero en la clave primaria de pedido. REGIONAL BY ROW AS region reparte las filas por esa columna. Al ir primero en PRIMARY KEY (region, pedido_id), las filas de una región quedan juntas y esa región puede ser su hogar. El costo es que una consulta que no filtre por region puede producir scatter.

Decisión 2: catalogo_producto es GLOBAL, no REGIONAL BY ROW. Es el catálogo pequeño que se quiere leer desde las tres regiones. GLOBAL da esa lectura en todas ellas; el precio es que la lectura global tiene costo de escritura. Con REGIONAL BY ROW habría que elegir una sola región hogar.

3. Réplica Raft frente a fragmentación y asignación

Diseño de fragmentación	En CockroachDB	No confundir
Fragmentación horizontal primaria por región	pedido: REGIONAL BY ROW AS region	hogar de fila ≠ consenso
Catálogo pequeño replicado	catalogo_producto: GLOBAL	lectura global tiene costo de write
Sitio / región	--locality=region=...	etiqueta ≠ WAN física
Réplica selectiva	política de colocación	réplica Raft copia un rango

Fragmentar y asignar son decisiones de diseño sobre relaciones: qué filas existen y en qué sitio viven. Una réplica Raft es otra cosa: una copia de un **rango**, mantenida por un grupo de consenso para sobrevivir a la caída de un nodo. Son fenómenos distintos en el mismo clúster. ti4601_raft.public.raft_probe no usa localidad multirregional y aun así tiene tres réplicas votantes; por eso la disponibilidad del §5 no debe atribuirse a REGIONAL BY ROW.

Varios rangos implican varios grupos Raft. El motor parte el espacio de claves en rangos y cada rango tiene sus propias réplicas, su líder Raft y su leaseholder: es una máquina de estados replicada independiente. En cluster-inspect.txt, raft_probe ocupa el rango 76 con sus votantes {1,3,2}; las filas de pedido viven en otros rangos, con sus propios grupos. El paper explica un log; el motor ejecuta muchos (Multi-Raft), y una consulta puede tocar varios rangos.

Límite de este clúster. REGIONAL BY ROW espera varios nodos por región y aquí hay uno, así que algunos rangos de pedido pueden aparecer subreplicados: la evidencia demuestra región hogar y localidad configurada, no residencia estricta.

4. Latencia: p50 y p99 de los cuatro casos

Gateway fijo crdb-1, warm-up de 5 descartado por caso y n=50 muestras por caso (200 en total; latency.csv tiene 201 líneas). El reloj es time.perf_counter_ns() y el p99 usa nearest rank.

operation	locality	home_region	n	p50_ms	p99_ms
read	local	cr-sj	50	0.690	1.934
read	remote	cr-limon	50	1.460	1.698
write	local	cr-sj	50	7.067	12.713
write	remote	cr-limon	50	6.072	11.792

Lo medido se reporta tal cual. En escrituras el caso remoto salió incluso por debajo del local: resultado válido, porque Docker no añade distancia física y la razón local/remoto puede quedar cerca de 1 o invertirse por ruido. La diferencia clara está entre lectura y escritura, no entre local y remoto.

Declaración obligatoria. (i) Las tres regiones son lógicas. (ii) La máquina, los recursos disponibles y el número de ejecuciones se mantuvieron constantes. (iii) Inferir el costo de una WAN requeriría latencia controlada, ausente hoy.

5. Escenario A: falla de un nodo, RTO y RPO

Precondición (chaos-a-before.txt). El rango 76 de raft_probe tenía voting_replicas {1,3,2}, una por nodo. El lease se movió al nodo que se iba a detener con ALTER RANGE RELOCATE LEASE TO 3 (el store_id de region=cr-limon), y quedó con lease_holder = 3. Ese node 3 es el contenedor ti4601-crdB-2: se detuvo el leaseholder, no un follower. El valor confirmado antes de la falla era version = 0.

Instante (UTC)	Evento	Fuente
17:37:40.543 – 17:37:47.039	14 escrituras before-stop ok	chaos-a.csv
17:37:47.5218	docker stop --timeout 0 ti4601-crdB-2	chaos-a-stop.epoch
17:37:49.539	after-stop error 2007.6 ms	chaos-a.csv
17:37:51.549	after-stop error 2009.2 ms	chaos-a.csv
17:37:51.5594	primer after-stop ok, 10.235 ms	chaos-a.csv
17:37:59.472	crdb-2 vuelve a estar arriba	chaos-a-node-status.txt
17:38:10.055	lectura posterior: version = 112	chaos-a-rpo.txt

RTO observado. Se toma el instante de falla de chaos-a-stop.epoch y la columna completed_epoch de la primera fila con phase=after-stop y status=ok:

$RTO = (1788370671.559400 - 1788370667.521754) \times 1000 = 4037.646 \text{ ms} \rightarrow 4037.6 \text{ ms}$

La sonda imprimió 4037.6 ms: el mismo cálculo, redondeado a un decimal.

Errores transitorios: 2, ambos QueryCanceled por el statement_timeout de 2 s de la sonda. Son intentos sin confirmación, de resultado desconocido; no se contabilizan como operaciones confirmadas perdidas.

RPO observado para operaciones confirmadas: 0. version solo se incrementa. El valor confirmado antes de la falla no retrocedió: la lectura posterior a la recuperación devuelve version = 112, mayor que el valor previo, y make lab1-status muestra los tres nodos con is_live = true.

6. Mayoría de 2 de 3

Con tres votantes la mayoría es 2, porque dos mayorías cualesquiera comparten al menos un servidor. Esa intersección es la que hace que un write committed sobreviva: el próximo líder necesita votos de una mayoría, y esa mayoría contiene a alguien que ya tiene la entrada. Mayoría no significa muchas copias; significa intersección.

La evidencia del §5 lo muestra: las escrituras volvieron a confirmar a las 17:37:51.56 y crdb-2 no regresó sino hasta las 17:37:59.47. Las 38 escrituras after-stop ok se confirmaron con dos nodos de tres, no porque el nodo caído volviera.

Con dos nodos caídos queda un proceso activo pero 1 de 3, menos que la mayoría: sigue vivo y acepta la conexión, pero no puede reunir mayoría para elegir líder ni para avanzar commitIndex, así que la escritura no debe confirmarse. Proceso activo no es lo mismo que quórum.

7. Puente al Proyecto 1

Se reutiliza el método, no los artefactos: Compose y la imagen cliente, las variables PG*, SHOW REGIONS, SHOW CREATE y SHOW RANGES, las definiciones de p50, p99, RTO y RPO, y la estructura precondición → evento → medición → verificación. Entregar este DDL renombrado no cumple el P1.

Queda pendiente y es trabajo propio del equipo: el dominio del E1 con sus predicados, claves y localidades, y su propia tabla REGIONAL BY ROW y su candidato a GLOBAL; un script reproducible de configuración, porque aquí se aplicó a mano en psql; un seed y un generador de carga propios; un programa que mida operaciones representativas del dominio; una tabla operacional propia con RF=3 para el E4, distinta de raft_probe; la orquestación de la falla y su evidencia; y el contraste del E5 contra un nodo o el Lab 0. Los límites que deben declararse: las regiones son lógicas, los rangos REGIONAL BY ROW pueden quedar subreplicados con un nodo por región, y de este Compose no se sigue ninguna afirmación sobre residencia de los datos.